

EXPERIMENT 8

Student Name: Aryan Dahiya

Branch: AIT-CSE (AIML)

Semester: 4

Subject Name: Database Management System

UID: 24BAI70697

Section/Group: 24AIT_KRG-G2

Subject Code: 24CSH-298

Experiment 8 – Design of a Parameterized PL/SQL Stored Procedure for Employee Count by Gender (CData, Toddle)

Experiment

Experiment 8: Designing and implementing a parameterized PL/SQL stored procedure that accepts gender as an input parameter and returns the total number of employees based on the given gender. This experiment demonstrates procedural programming, parameterized logic, and dynamic data retrieval in a database environment.

Aim

To design and implement a stored procedure that accepts gender as an input parameter and returns the total number of employees based on the gender passed, thereby demonstrating procedural programming and parameterized logic in PL/SQL.

Objective

- To understand the structure and components of a PL/SQL stored procedure.
- To implement IN and OUT parameters in stored procedures.
- To pass dynamic input values to procedures.
- To apply aggregate functions for data analysis.
- To develop reusable database logic for real-world business scenarios.

Software Requirements

Database Management System:

- Oracle Database Express Edition (Oracle XE)
- PostgreSQL Database

Database Administration Tool / Client Tool:

EXPERIMENT 8

- Oracle SQL Developer (for Oracle XE)
- pgAdmin (for PostgreSQL)

Problem Statement

Organizations often require flexible reporting mechanisms to analyze workforce distribution based on gender for compliance and internal analytics. A dynamic and reusable solution is needed to efficiently retrieve employee counts based on gender input.

Practical / Experiment Steps

- Create an employee table with relevant attributes.
- Insert sample data into the employee table.
- Design a stored procedure that accepts gender as an input parameter.
- Use an aggregate function to count employees based on the given gender.
- Return the result using an OUT parameter.
- Execute the stored procedure with different gender inputs.
- Analyze the output.

Procedure

1. Open Oracle SQL Developer or pgAdmin and connect to the database.
2. Create the employee table with fields such as id, name, and gender.
3. Insert sample records into the table.
4. Create a PL/SQL stored procedure with IN and OUT parameters.
5. Write a SELECT COUNT(*) query inside the procedure.
6. Compile the procedure successfully.
7. Execute the procedure using different gender values.
8. Display the output.
9. Capture results for documentation.

EXPERIMENT 8

Input / Output Details

Input

- Table:
 - employee (id, name, gender)
- Input Parameter:
 - Gender value (e.g., 'Male', 'Female')

Step-wise Output

Step 1 – Create Employee Table

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100) NOT NULL,  
    salary NUMERIC(10,2) NOT NULL,  
    department VARCHAR(50)  
);
```

Step 2 – Insert Sample Data

```
-- 2. Insert sample data  
INSERT INTO employees (emp_id, emp_name, salary, department) VALUES  
(1, 'Alice', 50000, 'HR'),  
(2, 'Bob', 60000, 'IT'),  
(3, 'Charlie', 55000, 'Finance');
```

Step 3 – Create Stored Procedure

```
CREATE OR REPLACE PROCEDURE get_Employee_Count_BY_Gender(IN P_GEN VARCHAR(20), OUT COUNT_EMP INT, INOUT STATUS VARCHAR)  
AS  
$$  
    BEGIN  
        SELECT COUNT(*) INTO COUNT_EMP FROM EMPLOYEES WHERE GENDER=P_GEN;  
        STATUS:='SUCCESS';  
    END;  
$$ LANGUAGE PLPGSQL
```

Step 4 – Execute Stored Procedure

EXPERIMENT 8

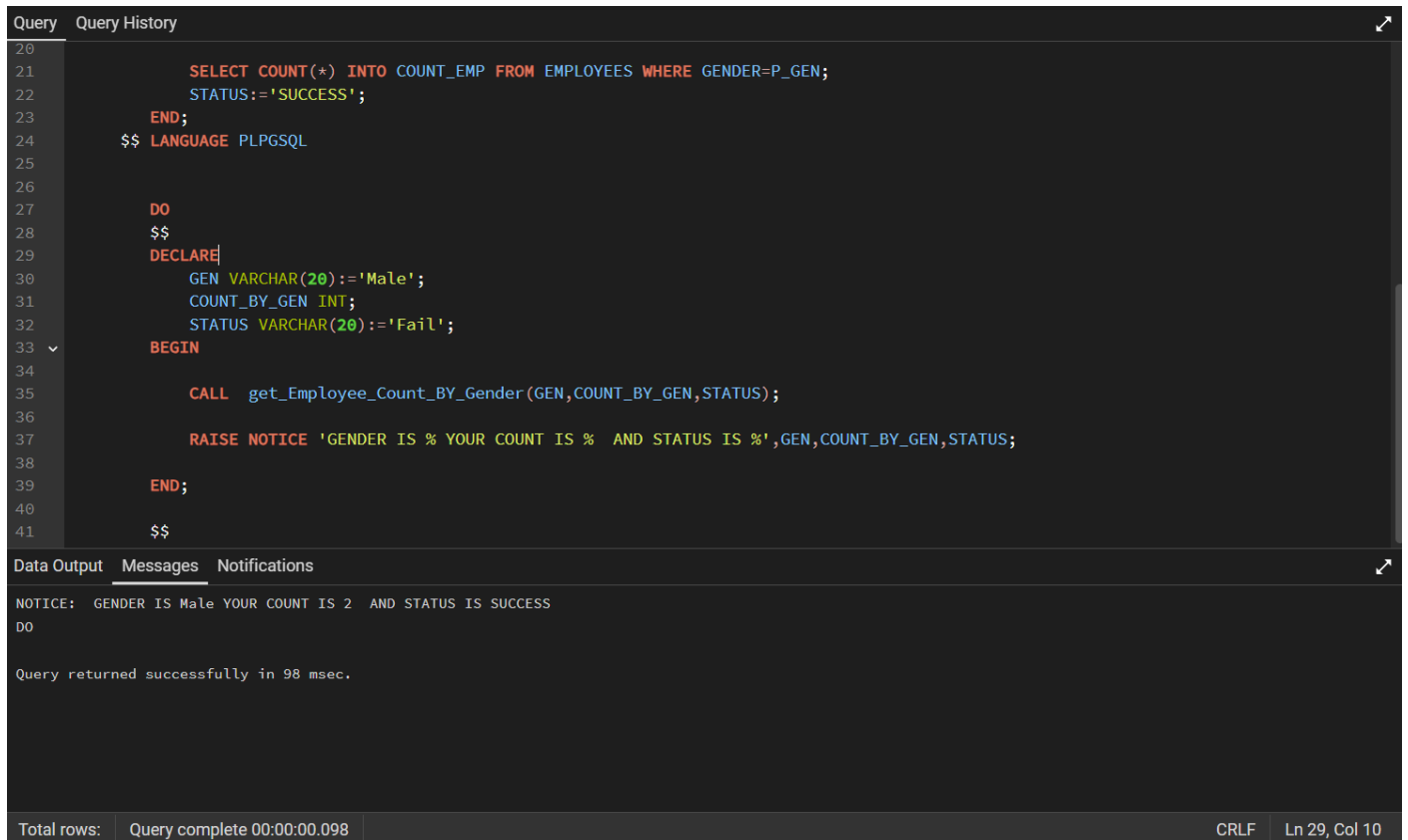
```
DO
$$
DECLARE
    GEN VARCHAR(20):='Male';
    COUNT_BY_GEN INT;
    STATUS VARCHAR(20):='Fail';
BEGIN

    CALL get_Employee_Count_BY_Gender(GEN,COUNT_BY_GEN,STATUS);

    RAISE NOTICE 'GENDER IS % YOUR COUNT IS % AND STATUS IS %',GEN,COUNT_BY_GEN,STATUS;

END;
$$
```

Step 5 – Display Output



The screenshot displays a database query editor with a dark theme. The top section, titled 'Query', shows a PL/SQL block being executed. The code defines variables for gender, count, and status, calls a procedure, and raises a notice. The bottom section, titled 'Data Output', shows the output of the notice: 'GENDER IS Male YOUR COUNT IS 2 AND STATUS IS SUCCESS'. The status bar at the bottom indicates 'Query complete 00:00:00.098' and 'Ln 29, Col 10'.

```
Query Query History
```

```
20
21      SELECT COUNT(*) INTO COUNT_EMP FROM EMPLOYEES WHERE GENDER=P_GEN;
22      STATUS:='SUCCESS';
23  END;
24  $$ LANGUAGE PLPGSQL
25
26
27  DO
28  $$
29  DECLARE
30      GEN VARCHAR(20):='Male';
31      COUNT_BY_GEN INT;
32      STATUS VARCHAR(20):='Fail';
33  BEGIN
34
35      CALL get_Employee_Count_BY_Gender(GEN,COUNT_BY_GEN,STATUS);
36
37      RAISE NOTICE 'GENDER IS % YOUR COUNT IS % AND STATUS IS %',GEN,COUNT_BY_GEN,STATUS;
38
39  END;
40
41  $$
```

Data Output Messages Notifications

```
NOTICE: GENDER IS Male YOUR COUNT IS 2 AND STATUS IS SUCCESS
DO

Query returned successfully in 98 msec.
```

Total rows: Query complete 00:00:00.098 CRLF Ln 29, Col 10

Learning Outcome

After completing this experiment, the learner will be able to:

- Understand procedural programming in PL/SQL.
- Implement parameterized stored procedures using IN and OUT parameters.
- Perform dynamic data retrieval based on user input.
- Use aggregate functions like COUNT() within procedures.

EXPERIMENT 8

- Develop reusable and efficient database logic for enterprise applications used in companies like CData and Toddle.